

SFC-009 — SFC as a Gap Bridging Algorithm

From Gap Detection to Governed Structural Extension

Sizhe Tan

with chatGPT (OpenAI)

2026-06-25

Repository: <https://github.com/sizhet/Structural-Feasibility-Confidence-SFC>

1. Introduction

Gap detection is only the beginning.

A system that can detect a gap has identified that something is missing, mismatched, incomplete, unsupported, or structurally disconnected.

But detection alone does not answer the next and harder question:

Should this gap be bridged, and if so, how?

Structural Feasibility Confidence (SFC) provides a way to turn gap detection into a governed gap-bridging algorithm.

The central idea is:

A gap is not merely an absence.

It is a candidate extension point whose bridgeability depends on its relation to confirmed tunnels, validated subgraphs, reusable modules, adjacent trajectories, mechanism understanding, and confidence thickness.

This changes the meaning of Gap Bridging.

Without SFC, Gap Bridging may become either too timid or too reckless:

- too timid if every gap is treated as a danger sign,
- too reckless if every gap is treated as a bridge target.

With SFC, the system can ask:

- What kind of gap is this?
- Where does it sit relative to known support?
- Is it inside the Confirmed Zone, Extension Zone, or Speculative Zone?
- What patchwork proof supports bridging?

- What policy is justified?

This document presents SFC as an algorithmic layer for Gap Bridging.

2. Gap Detection Is Not Gap Bridging

A detected gap may mean many different things.

It may be:

- a small missing operator,
- a missing task decomposition,
- a missing evidence link,
- a missing action path,
- a missing causal mechanism,
- a missing tool,
- a missing context condition,
- or a fundamental mismatch between target and capability.

These are not equivalent.

A system that treats all gaps the same will fail in one of two ways.

2.1 Over-bridging

The system tries to bridge gaps that lie outside supported territory.

This leads to:

- hallucinated plans,
- brittle implementations,
- false promises,
- uncontrolled exploration,
- and unsafe commitments.

2.2 Under-bridging

The system refuses gaps that are actually bridgeable.

This leads to:

- excessive conservatism,
- missed opportunities,
- weak generalization,
- and inability to grow capability.

SFC exists to reduce both errors.

3. The Core Principle

The core principle of SFC-based Gap Bridging is:

A gap should be bridged according to its structural support, not merely according to its existence.

This means the system must evaluate:

- proximity to confirmed structures,
- subtask coverage,
- module reuse,
- adjacent trajectory support,
- mechanism understanding,
- bridge cost,
- failure recoverability,
- and commitment risk.

A gap inside a thick confidence region may be bridged directly.

A gap near a boundary may require a probe or staged bridge.

A gap outside supported territory should not be treated as an ordinary extension.

4. Gap as Candidate Extension Point

Traditional gap language often emphasizes deficiency:

- something is missing,
- something is broken,
- something is incomplete.

SFC adds a second interpretation:

A gap is also a possible growth point.

A gap may identify where the system can extend a tunnel, create a new bridge, add a module, learn a pattern, or upgrade its future support field.

This is especially important for SCR and AI agent systems.

A mature runtime should not fear every gap.

It should learn to distinguish:

- gaps that should be bridged,
- gaps that should be probed,
- gaps that should be routed to other systems,
- and gaps that should be avoided.

This is what makes SFC an engine of controlled exploration.

5. A Basic SFC Gap-Bridging Algorithm

A simple SFC-informed Gap Bridging procedure can be described in nine steps.

6. Step 1 — Detect the Gap

The system first identifies a gap relative to a target.

The gap may appear as:

- a missing node,
- a missing edge,
- an unsupported claim,
- an unimplemented operator,
- an absent action path,

- a weak dependency,
- a broken analogy,
- or an unclear context condition.

The output of this step is not yet a bridge.
It is only a gap candidate.

7. Step 2 — Type the Gap

The system then classifies the gap.

Possible gap types include:

7.1 Action Gap

A concrete action operator is missing.

7.2 Task Gap

A task decomposition or task dependency is missing.

7.3 Evidence Gap

A claim lacks supporting evidence.

7.4 Mechanism Gap

The system lacks an explanation of why the transition should work.

7.5 Context Gap

The required condition, environment, or constraint is unclear.

7.6 Interface Gap

Two modules exist but do not yet connect cleanly.

7.7 Scale Gap

The pattern works at one scale but is untested at another.

7.8 Risk Gap

The action may be possible, but failure cost is too high or poorly bounded.
Gap typing is important because different gaps require different bridging policies.

8. Step 3 — Localize the Gap Relative to Support

The system asks where the gap sits relative to existing structures:

- confirmed tunnels,
- validated subgraphs,
- reusable modules,

- adjacent trajectories,
- known decomposition patterns,
- and mechanism-level knowledge.

This localization step prevents the system from treating the gap in isolation. A missing operator surrounded by validated modules is very different from a missing operator floating in unsupported space.

9. Step 4 — Estimate Structural Feasibility Confidence

The system then estimates the SFC of bridging the gap.

The gap may fall into one of three broad regions.

9.1 Confirmed-Adjacent Gap

The gap is close to already validated structures.

The bridge is likely routine or nearly routine.

9.2 Extension-Feasible Gap

The gap is not directly confirmed, but it is supported by patchwork proof and lies inside meaningful confidence thickness.

9.3 Speculative Gap

The gap lies beyond current support.

Bridgeability is unclear, weakly supported, or high-risk.

This classification is the heart of SFC-based Gap Bridging.

10. Step 5 — Build Patchwork Feasibility Proof

For a confirmed-adjacent or extension-feasible gap, the system should identify what supports the bridge.

Possible support sources:

- solved subtasks,
- reusable modules,
- neighboring trajectories,
- known operators,
- prior bridge patterns,
- mechanism understanding,

- fallback strategies,
- and partial tests.

The goal is not always to prove the entire target end-to-end.

The goal is to determine whether there is enough local support to justify an attempted bridge.

11. Step 6 — Select a Bridging Policy

Different SFC zones imply different policies.

11.1 Direct Bridge

Use when support is strong and failure cost is low.

11.2 Staged Bridge

Use when the bridge is likely feasible but contains nontrivial uncertainty.

11.3 Probe First

Use when a small experiment can cheaply reduce uncertainty.

11.4 Tool or Brain-Unit Escalation

Use when the gap is bridgeable but outside the current active capability cluster.

11.5 Scope Narrowing

Use when the full target is too large but a smaller target remains supported.

11.6 Defer

Use when support is too weak for current commitment.

11.7 Refuse

Use when the gap is speculative, high-risk, or unsupported by available structure.

This policy selection step is where SFC becomes operational.

12. Step 7 — Execute or Probe the Bridge

If the selected policy allows action, the system attempts the bridge.

The bridge may involve:

- generating a missing action path,
- connecting two modules,
- adding a task node,
- retrieving missing evidence,

- testing a partial implementation,
- asking a clarifying question,
- or creating a prototype.

The important point is that the attempt is governed by the SFC classification rather than by blind optimism.

13. Step 8 — Monitor Outcome

The system observes whether the bridge succeeds, partially succeeds, fails recoverably, or fails in a way that reveals deeper unsupported structure.

Relevant signals include:

- execution success,
- test results,
- human feedback,
- error patterns,
- cost overrun,
- rework requirements,
- or newly discovered dependencies.

Monitoring matters because gap bridging is not only about one attempt. It also produces information about the support field.

14. Step 9 — Update the Support Field

The final step is to update future SFC.

If the bridge succeeds:

- the bridge may become part of confirmed structure,
- confidence thickness may expand,
- a new module may be added,
- a new adjacent trajectory may become available.

If the bridge fails:

- the system may mark the region as thinner,
- add a caution boundary,
- reclassify similar gaps,
- or discover a hidden mechanism gap.

This update loop is essential.

It turns Gap Bridging from one-time improvisation into cumulative structural learning.

15. The Algorithm in Compact Form

The SFC Gap Bridging procedure can be summarized as:

1. Detect the gap.
2. Type the gap.
3. Localize the gap relative to confirmed tunnels and validated subgraphs.
4. Estimate SFC for bridging.
5. Build patchwork feasibility proof.
6. Select a bridging policy.
7. Execute, probe, escalate, narrow, defer, or refuse.
8. Monitor the outcome.
9. Update the support field.

This is not yet a full mathematical algorithm.

It is a structural runtime algorithm skeleton.

Its importance lies in the sequence:

Do not bridge merely because a gap exists.

Bridge according to structural support and commitment risk.

16. Why This Makes Us More Willing to Try

SFC does not only prevent reckless bridging.

It also gives a system more justified courage.

When a gap is supported by:

- nearby confirmed structures,
- reusable modules,
- adjacent trajectories,

- and strong patchwork proof,

the system can attempt the bridge more confidently.

This matters because many important discoveries and capability upgrades begin as gaps.

Without SFC, a system may avoid these gaps because they are not yet confirmed.

With SFC, the system can recognize that some gaps are precisely where controlled extension should happen.

In this sense, SFC does not make Gap Bridging timid.

It makes it **bolder in the right places** and **more cautious at the right boundaries**.

That is a major algorithmic contribution.

17. SFC and the Gap-Why Engine

SFC also strengthens the Gap-Why Engine.

A Gap-Why Engine detects missing structure and generates questions such as:

- Why does this transition fail?
- Why is this node unsupported?
- Why does this task not connect to an action path?
- Why is the result not explainable by the current structure?

SFC adds the next layer:

- Is this gap bridgeable?
- What kind of support surrounds it?
- Is this Why question likely to lead to a local bridge or a new research direction?
- How should the system allocate effort?

So SFC converts some "Why" outputs into bridge candidates with ranked policies.

This creates a stronger pipeline:

Gap detection

→ Why generation

→ SFC estimation

→ Patchwork proof

→ Bridging policy

→ Runtime update

This pipeline is especially important for SCR and future structural AI systems.

18. SFC and Task Calling Graph Growth

Task Calling Graphs are another major beneficiary.

A Task Calling Graph often contains missing or not-yet-confirmed action details.

Without SFC, such gaps may make the graph appear weak.

With SFC, the graph can be evaluated more precisely:

- which missing action paths are confirmed-adjacent?
- which are extension-feasible?
- which are speculative?
- which require new modules?
- which can be bridged by known orchestration patterns?

This means SFC becomes an algorithm for growing Task Calling Graphs.

The system can expand the graph not arbitrarily, but along supported extension directions.

19. SFC and LLM-Generated Bridges

LLMs can propose many possible bridges:

- missing text,
- missing code,
- missing task decomposition,
- missing explanation,
- missing analogy,

- missing implementation step.

But LLM-generated bridges should not be accepted merely because they are fluent.

SFC provides a governance layer:

- Does the proposed bridge have support?
- Is it confirmed, extension-feasible, or speculative?
- Does it require evidence retrieval?
- Does it require execution testing?
- Does it require human review?
- Should it be expressed as a hypothesis rather than a fact?

This is crucial for using LLMs as generators inside structural runtimes.

20. Risk and Boundary Discipline

SFC-based Gap Bridging also requires boundary discipline.

A gap may be technically bridgeable but operationally unsafe because:

- failure cost is high,
- recovery is difficult,
- consequences are irreversible,
- or the system lacks monitoring.

Therefore, bridgeability is not only a technical question.

It is also a risk-governance question.

A high-risk gap may require:

- human approval,
- staged testing,
- sandbox execution,
- simulation,

- or refusal.

This is another reason SFC must include commitment policy rather than only confidence scoring.

21. A Practical Classification Table

A simple working table can guide early implementations.

Gap Class	Support Level	Suggested Policy
Confirmed-adjacent	High	Direct bridge
Extension-feasible	Medium-high	Staged bridge or probe
Thin extension	Medium-low	Prototype first / escalate
Speculative	Low	Defer, narrow, or refuse
High-risk speculative	Low + high consequence	Refuse or require human governance

This table is not final, but it shows the direction.

The key idea is that gap class should control bridging policy.

22. Conclusion

SFC turns Gap Bridging from a vague ambition into a governed structural process.

It teaches that a gap should not be treated merely as a missing piece.

It should be evaluated as a candidate extension point relative to the system's current support field.

The resulting algorithm is:

- detect the gap,
- type the gap,
- localize it,
- estimate SFC,
- build patchwork proof,
- choose a bridging policy,
- attempt or defer,

- monitor outcome,
- update future confidence.

This gives a runtime two complementary strengths:

- more courage to try gaps that are structurally supported,
- more discipline to avoid gaps that are unsupported or high-risk.

That combination is central to future SCR, Task Calling Graph growth, LLM-governed generation, and AI agent runtime intelligence.

Key Takeaways

- Gap detection is not enough; a system must decide **whether and how** to bridge.
- SFC provides an algorithmic layer for Gap Bridging.
- A gap should be classified by:
 - type,
 - support,
 - zone,
 - bridgeability,
 - risk,
 - and commitment policy.
- SFC makes Gap Bridging both:
 - **bolder in supported extension zones**, and
 - **more disciplined near speculative boundaries**.
- A compact SFC Gap Bridging loop is:
 - detect,

- type,
- localize,
- estimate,
- prove patchwork support,
- select policy,
- execute/probe/escalate/refuse,
- monitor,
- update.
- SFC directly strengthens:
 - Gap-Why Engine,
 - Task Calling Graph growth,
 - LLM-generated bridge governance,
 - and SCR runtime learning.